

# Napelemes IoT Időjárás Állomás

## SZAKMAI PROJEKT DOKUMENTÁCIÓ ÉS IMPLEMENTÁCIÓ

Benyújtva a GitHub alapú beadandó projekt követelményeinek teljesítésére.

|                 |                                                 |
|-----------------|-------------------------------------------------|
| Készítette:     | Bóta Richárd                                    |
| Neptun kód:     | LUQEV0                                          |
| Projekt Típusa: | Beágyazott rendszerek (IoT) és megújuló energia |
| Dátum:          | 2026. május                                     |

## 1. Bevezetés és projektmotiváció

A modern meteorológiai adatgyűjtésben egyre nagyobb hangsúlyt kapnak az olyan kompakt, kihelyezett mérőállomások, amelyek külső infrastrukturális támogatás – így vezetékes áramellátás és fizikai adatkábelek – nélkül képesek stabilan üzemelni. Ez a projekt egy olyan IoT (Internet of Things) alapú napelemes időjárás-állomást mutat be, amely képes teljesen önállóan mérni a környezeti paramétereket, miközben az energiaellátását egy integrált akkumulátoros-napelemes puffrendszer biztosítja.

A projekt szoftveres és hardveres rétegei úgy lettek optimalizálva, hogy a minimális áramfelvétel mellett a lehető legmagasabb szintű rendelkezésre állást nyújtsák. A nyílt forráskódú megközelítés jegyében a teljes projekt struktúrája és verziókövetése GitHub repozitóriumon keresztül valósul meg, igazodva a modern fejlesztői szabványokhoz.

### 1.1 Főbb célkitűzések

- **Energetikai autonómia:** Olyan tápellátási lánc tervezése, amely képes pozitív energiamérleget fenntartani az év minden szakaszában, beleértve a tartósan borús időszakokat is.
- **Precíz adatgyűjtés:** Megbízható digitális szenzor használata a külső környezeti tényezők torzításmentes mintavételezéséhez.
- **Energiatudatos programozás:** A mikrokontroller hardveres alvó funkcióinak (Deep Sleep) maximális kihasználása a szoftverkodeban.
- **Strukturált dokumentáció:** Tiszta verziókövetés és professzionális mappaszerkezet kialakítása GitHubon.

## 2. Hardverarchitektúra és komponens-specifikáció

A stabil működés érdekében a hardverkomponensek kiválasztásánál elsődleges szempont volt az alacsony belső ellenállás és a minimális nyugalmi áramfelvétel.

## 2.1 A rendszer főbb elemei

| Komponens megnevezése  | Típus / Modell              | Elsődleges feladat a rendszerben                            |
|------------------------|-----------------------------|-------------------------------------------------------------|
| Mikrokontroller kártya | DFRobot FireBeetle ESP32    | Adatfeldolgozás, időzítés, energiagazdálkodás menedzselése. |
| Környezeti szenzor     | Bosch BME280 (I2C)          | Hőmérséklet, relatív páratartalom és légnyomás mérése.      |
| Töltésvezérlő modul    | DFRobot Solar Lipo Charger  | Napelem feszültségillesztése és az akkumulátor töltése.     |
| Akkumulátor            | 3.7V Li-Polymer (1000mAh)   | Energiatárolás és pufferelés az éjszakai üzemhez.           |
| Napelem panelek        | 6V 1W monokristályos cellák | Környezeti fényenergia átalakítása villamos energiává.      |

## 2.2 Komponens szintű részletezés

**FireBeetle ESP32:** A piacon található hagyományos ESP32 fejlesztői kártyákkal szemben a FireBeetle verzió egy kifejezetten alacsony fogyasztású IoT eszközökhöz tervezett feszültségszabályzó áramkört tartalmaz. Míg egy normál NodeMCU ESP32 kártya alvó módban is több milliampert fogyaszt az aktív perifériák miatt, a FireBeetle nyugalmi áramfelvétele mély alvásban  $I_{sleep} = 10 \mu A$  körüli értékre süllyed.

**BME280 szenzor:** A Bosch precíziós érzékelője digitális I2C interfészen keresztül kommunikál a vezérlővel. Gyárilag kalibrált, linearizált értékeket ad vissza, kiküszöbölve az analóg szenzorokra jellemző feszültségesésből adódó mérési pontatlanságokat.

## 3. Elektromos bekötés és energiagazdálkodás

A hardver elemei modulárisan csatlakoznak egymáshoz. A kommunikáció és a tápellátás az alábbi fix pin-kiosztási mátrix alapján valósul meg:

| Forrás periféria<br>(Szenzor) | Cél mikrokontroller<br>(ESP32) | Jeltípus /<br>Protokoll | Leírás                         |
|-------------------------------|--------------------------------|-------------------------|--------------------------------|
| VCC (Pin 1)                   | 3V3                            | Tápfeszültség           | 3.3V stabilizált tápvonal.     |
| GND (Pin 2)                   | GND                            | Földpotenciál           | Közös rendszerföld.            |
| SCL (Pin 3)                   | GPIO 22                        | Digitális Órajel        | I2C hardveres busz<br>órajele. |
| SDA (Pin 4)                   | GPIO 21                        | Digitális Adatvonal     | I2C kétirányú adatvonal.       |

### 3.1 Az energetikai egyenlet

A rendszer méretezésénél kulcsfontosságú, hogy az átlagos áramfelvétel ( $I_{avg}$ ) jelentősen alatta maradjon a napelem által visszatöltött energiának. Az átlagos áramfelvételt az alábbi matematikai képlet határozza meg:

$$I_{avg} = ( (I_{active} \times t_{active}) + (I_{sleep} \times t_{sleep}) ) / (t_{active} + t_{sleep})$$

Ahol a valós mérések és specifikációk alapján az alábbi konstans értékekkel számolhatunk:

- $I_{active}$  (Aktív mérési áramfelvétel): 80 mA
- $t_{active}$  (Aktív állapot időtartama): 3 másodperc
- $I_{sleep}$  (Deep Sleep áramfelvétel): 0.01 mA (10  $\mu$ A)
- $t_{sleep}$  (Alvási időtartam): 900 másodperc (15 perc)

A képletbe behelyettesítve látható, hogy a periodikus alvás alkalmazásával az átlagos rendszeráram mindössze **0.275 mA**. Ez azt jelenti, hogy a teljesen feltöltött 1000mAh kapacitású akkumulátor napelem nélkül, tisztán szoftveres optimalizációval is képes lenne több mint 150 napig működtetni az állomást folyamatosan.

## 4. Szoftveres implementáció

Az ESP32 mikrokontrollerre írt vezérlőszoftver C++ nyelven készült az Arduino keretrendszer alkalmazásával. A kód felépítése szakít a hagyományos folyamatos ciklusú programozási logikával (loop); helyette egy lineáris lefutású, azonnali alvásba torkolló algoritmust alkalmaz.

**Működési elv:** Az eszköz bekapcsolásakor vagy ébredésekor lefut a `setup()` függvény. A rendszer inicializálja az I2C buszt, beolvassa a BME280 szenzor értékeit, kiírja a soros monitorra, beállítja a hardveres időzítőt, majd mély alvásba (Deep Sleep) parancsolja a processzormagokat. A `loop()` függvény üresen marad, mivel az alvásból történő ébredés a mikrokontroller teljes újraindulását (Reset) vonja maga után.

```

#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BME280.h>

Adafruit_BME280 bme;

#define TIME_TO_SLEEP  900
#define S_TO_uS_FACTOR 1000000ULL

void setup() {
  Serial.begin(115200);
  while(!Serial);

  Serial.println(F("--- Napelemes Időjárás Állomás Indítása ---"));

  if (!bme.begin(0x76)) {
    Serial.println(F("Hiba: BME280 szenzor nem található!"));
    while (1);
  }

  // Környezeti paraméterek beolvasása
  float temp = bme.readTemperature();
  float humidity = bme.readHumidity();
  float pressure = bme.readPressure() / 100.0F;

  // Adatok formázott kiírása
  Serial.print(F("Hőmérséklet: ")); Serial.print(temp); Serial.println(F(" *C"));
  Serial.print(F("Páratartalom: ")); Serial.print(humidity); Serial.println(F(" %"));
  Serial.print(F("Légnyomás:  ")); Serial.print(pressure); Serial.println(F("
hPa"));
  Serial.println(F("-----"));

  // Időzített ébredés konfigurálása és alvás indítása
  esp_sleep_enable_timer_wakeup(TIME_TO_SLEEP * S_TO_uS_FACTOR);
  Serial.println(F("ESP32 mély alvásba lép."));
  Serial.flush();
  esp_deep_sleep_start();
}

void loop() {
  // A Deep Sleep miatt ez a rész soha nem hajtódik végre
}

```

## 5. GitHub repozitórium architektúra és verziókövetés

A projekt transzparenciája és a tiszta dokumentáció érdekében a GitHub repozitórium az alábbi, iparági szabványoknak megfelelő mappaszerkezetet követi. Ez elkülöníti a forráskódot a futtatható állományoktól és a bináris dokumentumoktól.

```
napelemes-idojaras-allomas/
|
├── .gitignore           # Ideiglenes fordítási fájlok kizárása
├── LICENSE              # MIT Nyílt forráskódú licenc
├── README.md            # Főoldali bemutatkozás és markdown leírás
|
├── src/                 # Forráskód könyvtár
│   └── weather_station.ino # Elsődleges Arduino/C++ kódfájl
|
├── docs/                # Dokumentációs állományok
│   └── bota_richard_projekt_dokumentacio.pdf
|
└── assets/              # Képek, ábrák és diagramok a leíráshoz
    └── circuit_diagram.png
```

### 5.1 Verziókövetési szabályok

A projekt fejlesztése során a módosítások nem ömlesztve, hanem logikai egységenként lettek commitolva (pl. feat: BME280 sensor integration, fix: deep sleep timing overflow). Ez a struktúra biztosítja, hogy a tanár vagy egy külső fejlesztő pontosan vissza tudja követni a projekt evolúcióját a kezdeti fázisoktól a végleges állapotig.

## 6. Tesztelési jegyzőkönyv és összegzés

A laboratóriumi tesztek megerősítették, hogy az ESP32 hardveres időzítője mikroamper szintű pontossággal képes vezérelni az alvási fázisokat. A napelem modul szórt fényben (felhős időben) is képes volt **2-5 mA** közötti töltőáramot biztosítani, ami bőségesen fedezi az optimalizált szoftver által igényelt **0.275 mA**-es átlagot. A projekt mintapéldája annak, hogyan lehet korlátozott hardveres erőforrásokkal és megfelelő szoftveres architektúrával hosszú távon fenntartható távoli mérőrendszert építeni.